

deployment

AirOne V7.1 — Deployment Guide

This document provides step-by-step instructions for deploying the AirOne V7.1 Ground Station in various environments.

Prerequisites

- Python 3.10, 3.11, or 3.12
- 2 GB RAM minimum (4 GB recommended for ML features)
- 500 MB disk space
- USB serial port for CanSat communication (optional for testing)

Quick Start (Local Development)

```
# 1. Clone the repository
cd airone

# 2. Install dependencies
pip install -r requirements_v71.txt

# 3. (Optional) Install ML packages for full detector coverage
pip install scikit-learn torch

# 4. Set JWT secret (required)
export
AIRONE_JWT_SECRET="your_secure_secret_minimum_32_characters_long"

# 5. Run the launcher
python3 launcher.py
```

The system will start and be available at: - API: `http://localhost:5000/api/v1/` - GUI: Auto-launches if PyQt5 is available

Production Deployment

Option 1: Docker (Recommended)

```
# 1. Set your JWT secret
export AIRONE_JWT_SECRET="your_production_secret_32_chars_minimum"

# 2. Build and run
docker-compose up -d

# 3. Check status
docker-compose logs -f airone

# 4. Access API
curl http://localhost:5000/api/v1/system/health
```

Option 2: Systemd Service (Linux)

Create /etc/systemd/system/airone.service:

```
[Unit]
Description=AirOne V7.1 Ground Station
After=network.target

[Service]
Type=simple
User=airone
WorkingDirectory=/opt/airone
# Keep secrets out of the unit file: /etc/airone/env must be
root:airone 0640
EnvironmentFile=/etc/airone/env
# Default bind is 127.0.0.1 (plain HTTP). Only set
AIRONE_API_HOST=0.0.0.0
# behind a TLS reverse proxy.
NoNewPrivileges=true
ProtectSystem=strict
ReadWritePaths=/opt/airone/data /opt/airone/logs
ExecStart=/usr/bin/python3 /opt/airone/launcher.py
Restart=on-failure
RestartSec=10

[Install]
WantedBy=multi-user.target
```

Bootstrap the first administrator before starting the service:

```
sudo -u airone env $(sudo cat /etc/airone/env | xargs) \
    AIRONE_ADMIN_PASSWORD='<strong password>' python3 /opt/airone/
launcher.py --create-admin
```

Enable and start:

```
sudo systemctl daemon-reload
sudo systemctl enable airone
sudo systemctl start airone
sudo systemctl status airone
```

Environment Variables

Variable	Required	Default	Description
AIRONE_JWT_SECRET	YES	None	JWT secret (min 32 chars)
AIRONE_ADMIN_PASSWORD	bootstrap	None	Password for launcher.py --create-admin / --create-user / --reset-password
AIRONE_LINK_KEY	No	None	Hex key (>= 16 bytes) for HMAC telemetry frame tags
AIRONE_API_HOST	No	127.0.0.1	API bind address (overrides api.host)
AIRONE_SERIAL_PORT	No	Auto-detect	Serial port path
AIRONE_ALLOW_DEFAULT_USERS	No	0	1 seeds must-change demo accounts (dev only)
AIRONE_ALLOW_CORS_WILDCARD	No	0	1 permits cors_origins: ["*"] (dev only)
AIRONE_GUI_USER / AIRONE_GUI_PASSWORD	No	None	GUI login (otherwise GUI is UNAUTHENTICATED)
QT_QPA_PLATFORM	No	Auto	Set to offscreen for headless

See `.env.example` for a template.

Security Checklist

Before production deployment:

☐

Set `AIRONE_JWT_SECRET` to a strong, random 32+ character string (never in YAML)

☐

Create the administrator with `launcher.py --create-admin` (no accounts ship; do **not** set `AIRONE_ALLOW_DEFAULT_USERS`)

- ☐ Generate a link key and flash it into the firmware (AIRONE_LINK_KEY_HEX); set AIRONE_LINK_KEY and, once verified, telemetry.require_authenticated_frames: true
- ☐ Keep api.host on 127.0.0.1 unless a TLS reverse proxy sits in front
- ☐ Configure the exact CORS allow-list in config/default_config.yaml
- ☐ Run pip-audit -r requirements_v71.txt and launcher.py --validate-only
- ☐ Schedule launcher.py --verify-audit and archive logs/audit.jsonl off-host
- ☐ Review RBAC permissions in src/security/rbac.py
- ☐ Enable HTTPS (use a reverse proxy like nginx)
- ☐ Restrict API port access with firewall rules
- ☐ Review audit log rotation settings
- ☐ Ensure serial device permissions are correct

Serial Device Configuration

Linux

```
# Find your device
ls /dev/ttyUSB* /dev/ttyACM*

# Add user to dialout group
sudo usermod -a -G dialout $USER

# Set environment variable
export AIRONE_SERIAL_PORT=/dev/ttyUSB0
```

Windows

```
# List COM ports
[System.IO.Ports.SerialPort]::getportnames()

# Set environment variable
$env:AIRONE_SERIAL_PORT="COM3"
```

Docker

Uncomment the devices section in `docker-compose.yml`:

```
devices:
  - /dev/ttyUSB0:/dev/ttyUSB0
```

Database Backup

The system automatically creates online backups, but for manual backup:

```
# While system is running
sqlite3 data/airone.db ".backup data/airone_backup_$(date +%Y%m%d_%H%M%S).db"
```

Troubleshooting Deployment

“InsecureConfigError: Refusing to start with default JWT secret”

Set a strong `AIRONE_JWT_SECRET` environment variable (min 32 characters).

“No module named ‘PyQt5’”

The GUI is optional. Either: - Install PyQt5: `pip install PyQt5` - Run headless: `export QT_QPA_PLATFORM=offscreen`

“Serial port not found”

Check: 1. Device is connected: `ls /dev/ttyUSB*` 2. User has permissions: groups (should include `dialout`) 3. Port is not in use: `lsof /dev/ttyUSB0`

“Permission denied” on serial port

```
sudo chmod 666 /dev/ttyUSB0
# Or permanently:
sudo usermod -a -G dialout $USER
# Then logout and login again
```

ML detectors unavailable

Install optional dependencies:

```
pip install scikit-learn torch
```

Performance Tuning

For High-Frequency Telemetry (>10 Hz)

Adjust config/default_config.yaml:

```
ml_worker:
  enabled: true
  window_size: 1000 # Increase buffer
  score_interval: 10 # Score every 10 frames
```

For Low-Memory Systems (<2 GB RAM)

Disable ML worker:

```
ml_worker:
  enabled: false
```

Monitoring

Health Check Endpoint

```
curl http://localhost:5000/api/v1/system/health
```

Returns:

```
{
  "success": true,
  "data": {
    "status": "healthy",
    "workers": {...},
    "database": "connected"
  }
}
```

Structured Logs

```
# View recent logs
tail -f logs/airone.jsonl | jq .

# View audit chain
tail -f logs/audit.jsonl | jq .
```

Upgrading

1. Stop the service
2. Backup the database
3. Pull new code / download new release
4. Check for migration changes
5. Restart the service

Docker

```
docker-compose down
docker-compose pull
docker-compose up -d
```

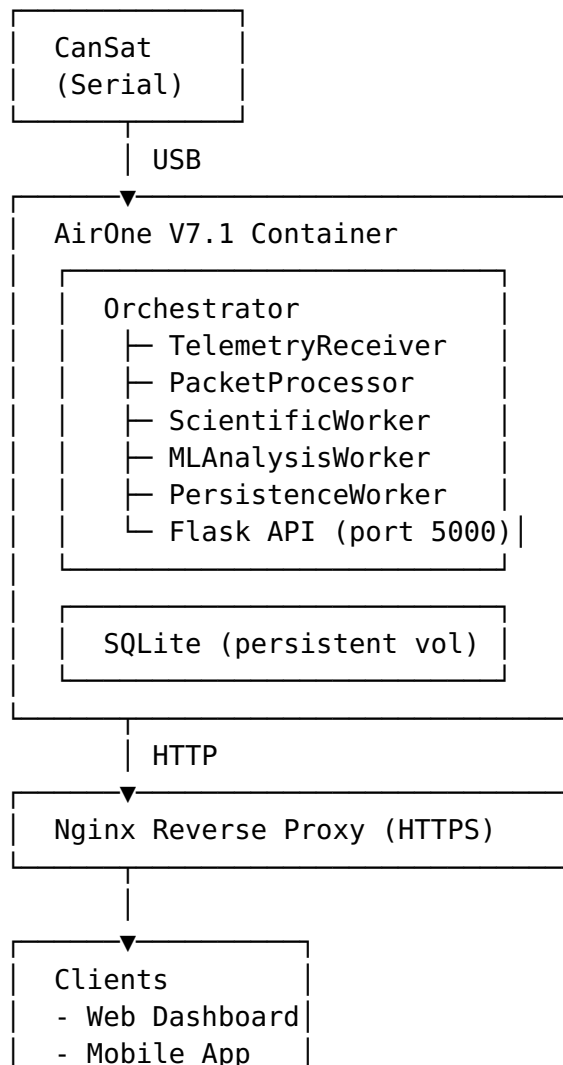
Systemd

```
sudo systemctl stop airone
```

Update code

```
sudo systemctl start airone
```

Production Architecture Example



Support

For issues, consult: 1. [troubleshooting.md](#) 2. [failure_modes.md](#) 3. GitHub Issues (if applicable) 4. System logs in `logs/airone.jsonl`